Отчёт по индивидуальному заданию №2

Архитектура компьютера и операционные системы

Вариант 24 «Танковые бои»

Выполнил: Махмутов Марсель Наилевич
Группа: 252

2026

Условие и устройство программы

Вариант 24, «Танковые бои». На клеточном поле действуют танки двух сторон: они наблюдают цели, перемещаются и стреляют. На каждом такте стороны независимо выбирают действия по одному состоянию поля, затем выполняют их по общим фазам. Порядок обхода участников не должен давать преимущества. Требуется консольная модель с выводом событий и итоговой статистики.

Программа написана на C для Linux с использованием libc, системных вызовов и библиотеки pthread. За основу взята программа из ДЗ1. Работает один процесс: каждому танку и координатору стороны соответствует отдельный поток. При N танках вместе с главным работают N + 3 потока: во встроенном примере 7, при 32 танках 35. Если бой завершён уже в начальном состоянии, рабочие потоки не создаются.

Сущность	**Представление и поведение**
Танк	`Tank`: сторона, номер, координаты, прочность, дальность. Поток `tank_main` вызывает `tank_plan`, выбирая движение и цель
Координатор стороны	`Team`: стратегия и статистика. Поток `team_main` выдаёт задания своим действующим танкам и собирает ответы
Снаряд	`Shot`: стрелявший танк, клетка цели, точность и такт прилёта. Обрабатывается главным потоком как отложенное событие
Поле и состояние боя	`Battle`: карта, массивы танков и снарядов, стороны и текущий такт

В `main.c` находятся запуск, общий цикл и завершение; в `config.c` чтение и проверка данных; в `battle.c` правила боя; в `output.c` карта и итоги; в `threads.c` работа потоков и синхронизация. Используются массивы: поле до 40×20, до 32 танков, полёт снаряда до 20 тактов. Запас 672 снаряда учитывает новые выстрелы до обработки прилетевших.

Потоки создаются один раз перед боем через `pthread_create`; аргументы передаются через структуры `ThreadArg`. Семафоры `start` уведомляют танки и координаторов о начале работы на очередном такте. Семафоры `done[0]` и `done[1]` учитывают ответы танков своих сторон, `done[2]` учитывает два ответа координаторов. Мьютекс защищает флаги остановки и ошибки, запись и копирование намерений. Сам расчёт выполняется вне мьютекса, параллельно по неизменному полю. Главный поток изменяет поле после ответа обеих сторон.

Правила и порядок такта

Координаты начинаются с нуля. Стратегия hold сохраняет позицию, advance приближает танк к ближайшему противнику до дальности стрельбы. Расстояние равно сумме разностей координат по модулю. За такт возможен переход в соседнюю клетку вверх, вправо, вниз или влево; при равенстве выбирается первое направление. Занятые в начале такта клетки и препятствия недоступны. Если несколько танков выбрали одну клетку, все остаются на месте.

Танк стреляет не чаще одного раза за такт по ближайшей цели в пределах дальности, при равенстве выбирает меньший номер. Стрельба навесная: препятствия не закрывают цели. Снаряд летит в выбранную клетку; при flight=1 выстрел первого такта прилетает на втором. Вероятность точного выстрела задаётся в процентах. Попадание снимает 1 единицу прочности у танка в клетке, в том числе союзного. Пустая клетка и неточный выстрел не дают урона. Уничтоженная машина остаётся препятствием.

1. Координаторы дают задания своим действующим танкам и ожидают их ответы. Танки наблюдают поле, выбирают движение и цели; поле ещё не меняется. После получения всех ответов координаторы сообщают главному потоку о готовности.

2. Регистрируются выстрелы из начальных позиций, разрешаются конфликты движения, танки перемещаются.

3. Обрабатываются прилетевшие снаряды. Урон сначала накапливается, затем одновременно применяется ко всем танкам.

4. Проверяется корректность состояния, выводится поле и определяется завершение.

Так сохраняются независимость планирования, однозначность конфликтов и запрет двух действующих танков в одной клетке. Прочность не опускается ниже нуля. Уничтоженные танки больше не действуют, но их выпущенные снаряды продолжают полёт до конца боя.

Интерфейс, запуск и завершение

Сборка и запуск из каталога проекта:

```
cmake -S . -B build
cmake --build build
./build/tanks
./build/tanks --config data/demo.txt --seed 24 --delay 200 --log battle.log
./build/tanks --config data/conflict.txt --limit 0
./build/tanks --help
```

Без параметров используется поле 12×8 с двумя танками каждой стороны. `-config` выбирает сценарий; `-seed` задаёт начальное значение генератора случайных чисел; `-delay` задержку между тактами в миллисекундах; `-limit` предел тактов, 0 означает отсутствие предела; `-log` файл итогов. Аргументы имеют приоритет над файлом.

Пример файла данных: `data/demo.txt`. Обязательны field и map. После map идут строки поля, затем записи `tank <A|B> <номер> <x> <y> <прочность> <дальность>`. При ошибках разбора сценария указывается строка.

На экране отображаются наблюдение, намерения, споры за клетки, движение, выбор цели, выстрелы, попадания, урон, уничтожение и поле после такта. Символы: . свободная клетка, # препятствие, A/B танки, X обломки.

Чтение использует open, read, close, вывод выполняется через dprintf по дескрипторам. В battle.log либо файл из `-log` записываются причина завершения, число тактов, оставшиеся танки, потери, выстрелы, попадания, выстрелы без урона и снаряды в полёте.

Обычное завершение наступает при победе, одновременном уничтожении сторон или достижении предела. Пустое поле сразу даёт ничью. В режиме `-limit` 0 возможен бесконечный бой, например спор за клетку в conflict.txt. Ctrl+C (SIGINT) и SIGTERM прерывают и конечный, и бесконечный режимы. Обработчик только устанавливает флаг. Начатый такт завершается перед остановкой, ожидание задержки прерывается.

Сигналы остановки принимает главный поток. При завершении он устанавливает общий флаг под мьютексом, пробуждает участников и ожидает их через `pthread_join`. Затем освобождает семафоры и мьютекс, сохраняет итоги и закрывает журнал. Потоки уничтоженных танков ожидают общего завершения. При неполном запуске освобождаются созданные ресурсы. Коды выхода: 0 при обычном завершении, 130 для SIGINT, 143 для SIGTERM; ошибки дают 1. Оставшиеся снаряды после конца боя не обрабатываются.

Проверки и оценка реализации

Проверки выполнены на Linux с GCC 11.4.0. Результаты в таблице получены с seed=24 и задержкой 0. Команда: `bash tests/run_tests.sh ./build/tanks`. Результат: `OK: 77 проверок пройдено.` Данные находятся в data, скрипт в tests/run_tests.sh.

Сценарий	**Полученный результат**	**Тактов**
demo.txt	Победа B; A: 0 живых, B: 1	7
win_a.txt, win_b.txt	Победа соответствующей стороны	2
draw.txt	Оба танка уничтожены, ничья	2
multi_hit.txt	Два попадания в B, победа A	2
conflict.txt, miss.txt	Предел: спор за клетку / точность 0	3
blocked.txt, wreck.txt	Предел: препятствие / обломки мешают движению	4
escape.txt	Предел: цель уходит из клеток обстрела	4
empty.txt	Ничья без танков	0
bad_map.txt, bad_overlap.txt	Ошибка длины строки / два танка в клетке; код 1	Не начат

Скрипт также проверяет повторяемость при одном seed, перестановку записей танков, ошибочный ввод, защиту исходного файла от перезаписи журналом, SIGINT и SIGTERM в обоих режимах. Повторные запуски проверяют воспроизводимость при работе нескольких потоков. Через `/proc` проверяется число потоков: 5 для двух танков и 7 для четырёх. Также проверяется прерывание без задержки. При 32 танках и полёте 20 после 50 тактов остаётся 640 снарядов, массив не переполняется.

Helgrind не обнаружил ошибок на сценариях с 4 и 32 танками. Искусственные отказы при создании потоков и семафоров обработаны без зависаний: 15 проверок.

Ограничения: движение не ищет обход препятствий; rand() может давать другие результаты в другой libc; ошибки вывода событий во время боя не проверяются, поэтому при закрытии получателя программа может продолжить работу без видимого вывода. Для застрявших танков доступны предел тактов и прерывание. Случайные результаты выстрелов определяет главный поток. Он же выводит намерения в порядке танков, поэтому порядок выполнения рабочих потоков не влияет на вывод при одинаковых исходных данных и seed в одной libc.